°C

Aare Temperature Daily Email

Project Description & Technical Documentation

An automation that checks the daily peak water temperature of the river Aare in Bern (the Marzili swimming spot) and emails a formatted daily report to a list of recipients every day at **18:00 Zurich time** — chosen because it falls after the warmest part of the day, so the reported peak reflects the true daily maximum. The whole system runs in the cloud; no personal computer needs to be switched on.

Property	Value
Recipients	kevin@holden.ch, jessica@holden.ch, brigitte.lendl@bluewin.ch
From address	aare@holden.ch (verified holden.ch domain)
Schedule	Daily at 18:00 Europe/Zurich
Repository	github.com/kejh-z/aare-temp (branch: main)

1. What the Email Contains

- **Peak water temperature** reached that day, and the time it occurred
- **Current water temperature** at the moment the report is generated
- **Flow rate** of the river (m³/s)
- **Number of measurements** taken that day
- A friendly **swim verdict** based on the peak temperature

Swim verdict thresholds:

Peak temperature	Verdict
≥ 20 °C	Perfect for swimming!
≥ 18 °C	Warm enough for a swim.
≥ 16 °C	A bit fresh, but doable.
< 16 °C	Too cold for most swimmers.

The email is delivered as styled HTML: a blue temperature card, a stats table, and a data-source footer.

2. Data Source

Website: **aaremarzili.ch** (redirects to aaremarzili.info)

Data API:

```
https://aareguru.existenz.ch/v2018/current?app=aare-temp&version=1.0.0
```

The aaremarzili site loads its data dynamically, so instead of scraping the page we use the public **aare.guru API** — the same underlying data source for Aare conditions in Bern. The API returns JSON containing:

- `aare.temperature` — current water temperature
- `aare.flow` — current flow rate
- `aarepast[]` — ~48 hours of historical readings (Unix timestamp + temperature, 10-minute intervals)

The app filters `aarepast[]` to readings from the current day (Zurich time), then finds the maximum temperature and the time it occurred.

3. How It Runs (Architecture)

Scheduling and execution are split across three services:

Stage	Service	Role
1. Scheduler	cron-job.org	Fires daily at 18:00 Zurich; sends an authenticated POST to the GitHub API to trigger the workflow.

2. Compute	GitHub Actions	Runs an inline Node.js script: fetches Aare data, computes the peak, builds the email, calls Resend.
3. Delivery	Resend	Sends the HTML email from aare@holden.ch to the recipients.

Trigger request sent by cron-job.org:

```
POST https://api.github.com/repos/kejh-z/aare-temp/
    actions/workflows/daily-email.yml/dispatches
Authorization: Bearer <GitHub personal access token>
Accept: application/vnd.github+json
Content-Type: application/json
Body: {"ref":"main"}
```

Why this combination?

- **GitHub's built-in cron was unreliable** — scheduled runs on inactive repos were delayed by hours or skipped. The `schedule:` trigger was removed and replaced with cron-job.org firing a `workflow_dispatch` event on time.
- **GitHub Actions** provides free, always-on compute, so no personal PC is needed.
- **Resend** handles deliverability and lets us send from a custom domain.

4. Reliability: IP Blocking & Self-Healing Retry

The aare.guru server (existenz.ch) runs **Imunify360 bot-protection**, which blocks a large share of cloud / datacenter IP addresses with the response `{"message":"Access denied by Imunify360 bot-protection..."}`. GitHub Actions runners use such IPs, and each run gets **one IP for its whole lifetime**. In practice roughly 40% of runs land on a blocked IP — the cause of the intermittent failures.

Because every request inside a run shares that one IP, retrying *within* a run (or sleeping and retrying) cannot escape a block. The fix is to retry as **separate runs**, each getting a fresh runner and therefore a fresh IP:

- A run fetches the data (3 quick attempts, ~10 s, to absorb genuine transient blips) and sends the email. On success it finishes.
- On failure, a second step (`if: failure()`) waits 90 s and uses the `GH_PAT` secret to dispatch a brand-new run with an incremented `attempt` counter.
- This repeats until a run succeeds or the cap of **6 attempts** is reached. The chain stops the instant one run succeeds, so recipients never get a duplicate.

With ~60% success per IP, six independent attempts give roughly **99.6%** reliability, and on a bad-IP day the email still typically arrives within a few minutes. GitHub's built-in token deliberately cannot trigger a `workflow_dispatch` (anti-recursion), which is why a separate `GH_PAT` token secret is required. A browser User-Agent is also sent, though the dominant factor is the IP, not the user-agent. Local runs via `index.js` use a residential IP that is not blocked, so they need no retry logic.

Verified end-to-end (2026-06-20) with a temporary `force_fail` switch (since removed): a forced failure auto-dispatched a fresh run on a new IP, and the chain stopped cleanly at the 6-attempt cap with no duplicate emails.

5. Repository Contents

File	Purpose
`.github/workflows/daily-email.yml`	**The live production code.** Self-contained GitHub Actions workflow with an inline Node.js script (no dependencies, no checkout). Runs each day.
`index.js`	Standalone Node.js version of the same logic for local testing. Uses the resend npm package and a .env file. Supports a --dry-run flag.
`package.json`	Declares the resend dependency and npm start / npm test scripts.
`.env.example`	Template showing the expected RESEND_API_KEY variable.
`.env`	(Not committed) Holds the real Resend API key for local runs.
`.gitignore`	Excludes node_modules/ and .env from git.
`setup-task.bat`	Legacy Windows Task Scheduler helper (superseded by the cloud setup).
`PROJECT.md`	Markdown source of this documentation.

Two copies of the logic — why?

The workflow contains an **inline** copy made self-contained (Node's built-in `fetch` + the Resend REST API directly) because the GitHub organisation restricts external Actions like `actions/checkout`, and a private repo could not be cloned without auth. `index.js` is the cleaner original kept for local testing. The two are functionally equivalent — if you change recipients or design, update **both** (the workflow is what runs in production).

6. Key Processes & Making Changes

Change recipients

Edit the `RECIPIENTS` array in **both** the workflow and index.js, then commit and push.

Change the send time

Adjust the schedule in cron-job.org (not in the repo). Keep the timezone set to Europe/Zurich.

Test manually

- **From GitHub:** Actions tab → "Daily Aare Temperature Email" → "Run workflow".
- **From cron-job.org:** use the "Test run" button.
- **Locally:** `npm test` (dry run) or `npm start` (sends a real email); requires a valid .env.

Rotate the API key

Create a new key in Resend, then update the `AARE_TEMP` secret at github.com/kejh-z/aare-temp/settings/secrets/actions.

7. Git Usage

- Git was initialised locally and pushed to GitHub (github.com/kejh-z/aare-temp).
- Default branch renamed from `master` to `main` (GitHub expects main; the Actions UI was not surfacing the workflow under master).
- Each change (adding recipients, switching domains, fixing the workflow) was a small, separate commit with a descriptive message.
- Secrets are never committed — .env is git-ignored and the production key lives only as a GitHub Actions secret.

8. Accounts & Credentials

Service	Purpose	Notes
GitHub (kejh-z)	Hosts the repo and runs the workflow	Repo: aare-temp
Resend	Sends the emails	holden.ch verified; key stored as the AARE_TEMP secret
cron-job.org	Triggers the workflow at 18:00	Uses a GitHub personal access token (classic, repo scope)

GitHub repository secrets:

Secret	Purpose
AARE_TEMP	The Resend API key, used to send the email.
GH_PAT	A GitHub token (classic, repo scope) used to re-dispatch a fresh run on failure. Can be the same token value cron-job.org uses.

9. Known Considerations & Gotchas

- **The data provider blocks cloud IPs (Imunify360)** — the single biggest source of failures. Handled by the self-healing retry (section 4): fresh runs on new IPs. Retrying within one run is useless, since its IP is fixed.
- **Re-dispatch needs the GH_PAT secret** — without it the workflow can only try once per trigger (the failure step logs that the secret is missing). GitHub's built-in token cannot start a new run.
- **GH_PAT must hold a complete, valid token** — a wrong or truncated value makes the re-dispatch return `{"message":"Bad credentials"}` and no retry spawns. The failed run still prints a "Dispatched fresh run" line regardless, so check the failure-step log for `Bad credentials` if retries are missing. (This bit us once; re-entering the exact token fixed it.)
- **The re-dispatch call is itself a network hop that can glitch** — on 2026-07-24 the first run hit the usual blocked IP, but the retry's curl died with `curl: (92) HTTP/2 stream ... CANCEL`, so no fresh run launched and no email went out that day. Fixed by forcing `--http1.1` and adding `--retry 5 --retry-all-errors` so a transient blip on the dispatch call can no longer break the chain.

- **cron-job.org header formatting matters** — the Authorization value must be exactly `Bearer <token>`. A truncated token caused a 401 during setup.
- **GitHub cron is unreliable** — intentionally not used; a duplicate email at ~22:16 was a late-firing GitHub cron run before its `schedule:` trigger was removed.
- **Resend free tier** originally only allowed sending to the account owner until the holden.ch domain was verified. Now any recipient works.
- If emails land in **spam**, sending separate emails per recipient (instead of one with all three in the To field) can improve deliverability — easy to switch on if needed.